BCSE307L · PROJECT A24 · TEAM 8 · BURS-BASED RETARGETABLE INSTRUCTION SELECTION

# Code guide: which file does what

Every file in the repository, its job, its key functions with line numbers, and how the files connect.
github.com/ahmadazim542/burs-instruction-selector

## 1. The whole repository on one page

```
burs-instruction-selector/
├── Makefile                builds everything in 3 stages (burg → generate → link)
├── README.md               overview, commands, results, demo walkthrough
├── .gitignore              keeps build/ out of git
├── src/                    ← the team's code
│   ├── ir/
│   │   ├── ir.h            the IR node type: the contract every module shares
│   │   └── ir.c            builds, prints, parses and evaluates IR trees
│   ├── burs/
│   │   ├── target.h        what a "target machine" looks like to the engine
│   │   ├── engine.h        the engine's public functions
│   │   └── engine.c        the BURS engine: label, cost, cover, emit (+ helpers)
│   ├── targets/
│   │   ├── targetA/targetA.gr  Target A: register machine (rules, emitter, simulator)
│   │   └── targetB/targetB.gr  Target B: stack machine (rules, emitter, simulator)
│   └── main/
│       └── main.c          the burs program: command line, reports, suite, tests
├── tests/
│   ├── cases.txt           31 regression cases: id | options | expression
│   ├── expected/*.out      the exact output each case must produce (31 files)
│   ├── run_tests.sh        runs the cases + suite + exhaustive check (make test)
│   └── make_results.sh     regenerates results/ (make results)
├── results/                saved evidence: rule tables, sample runs, suite, tests, benchmark
├── docs/                   Review 1–3 documents, slides, notes, guidelines
├── third_party/burg/       BURG generator (not our code; built by the Makefile)
└── build/                  created by make, not in git: burg, gen/targetA.c, gen/targetB.c, burs
```

### How one run flows through the files

What happens when you type `./build/burs "a + b * c"`:

**1. Command line**
main.c reads options and the expression
`main()`

**2. Parse**
ir.c turns text into an IR tree
`ir_parse()`

**3. Label**
engine.c asks the target's generated matcher to label every node
`burs_label()`

**4. Cover**
engine.c reads the best rules back from the root
`burs_print_cover()`

**5. Emit**
engine.c walks the cover; the target's .gr turns each rule into instructions
`burs_reduce()` → `reduceA()`

**6. Check**
the target's simulator runs the code; main.c compares with ir.c's value
`simulateA()` vs `ir_eval()`

Grey: machine-independent files (same for every target). Amber: target-specific code in `targetA.gr` / `targetB.gr`. Steps 3–6 repeat for Target B on the same tree.

### How the program is built (`make`)

| Stage | Command (from the Makefile) | Result |
|---|---|---|
| 1 | `cc … -o build/burg third_party/burg/*.c` | The BURG generator program. |
| 2 | `build/burg -p burmA -o build/gen/targetA.c targetA.gr`<br>`build/burg -p burmB -o build/gen/targetB.c targetB.gr` | Two C files: each is a generated matcher (state tables + `label`, `rule`, `nts`, `kids`) followed by the grammar's own C code, copied unchanged. The prefixes stop the two from clashing. |

| | | |
|---|---|---|
| 3 | `cc -std=c99 … -o build/burs ir.c engine.c main.c gen/targetA.c gen/targetB.c` | The single `burs` program containing both targets. |

## 2. Source files, one by one

### `src/ir/ir.h`

88 lines · IR module **Vineet**

Defines the intermediate representation: the tree node every other file uses. It is the single contract between the parser, the engine and both targets.

| Item | Line | What it is |
|---|---|---|
| `enum { CONST=1 … DIV=7 }` | 14 | The seven IR operators (DIV was added last). The numbers must match the `%term` line in both grammars. |
| `struct node` | 29 | One tree node: `op`, `state_label` (written by the matcher), `id` (n1, n2…), `ival` (constant), `name` (variable), `left`, `right`. |
| `OP_LABEL`, `STATE_LABEL`, `LEFT_CHILD`, `RIGHT_CHILD` | 39–42 | Accessor macros the burg-generated matchers use to read and label the tree. |
| function prototypes | 44–86 | Declarations of everything in `ir.c` (builders, printers, parser, evaluator). |

**Used by:** every other source file.

### `src/ir/ir.c`

419 lines · IR module / front end **Vineet**

Everything about IR trees that does not depend on any machine: building nodes, printing them, parsing text into a tree, and computing the "correct answer" used to check generated code.

| Function | Line | What it does |
|---|---|---|
| `ir_opname`, `ir_arity` | 18, 23 | Name ("ADD") and number of children of an operator. |
| `mk_const`, `mk_var`, `mk_add`, `mk_sub`, `mk_mul`, `mk_div`, `mk_load` | 46–64 | Tree builders: allocate one node each. |
| `freetree` | 66 | Frees a whole tree. |
| `ir_number` | 85 | Numbers nodes n1…nN in postorder (the ids shown in covers and cost tables). |
| `printtree`, `ir_draw` | 103, 144 | One-line prefix form `ADD(VAR a, MUL(…))` and the indented tree drawing. |
| `parse_primary`, `parse_term`, `parse_expr` | 213, 278, 294 | Recursive-descent parser: numbers, names, `( )`, `[e]` / `load(e)`; `*` and `/` before `+` `-`; left-associative. Produces the error messages ("column 7: expected ')'…"). |
| `ir_parse` | 310 | Entry point: text → tree, or `NULL` plus an error message. |
| `ir_set_var`, `ir_var_value`, `ir_mem_read` | 354, 370, 385 | Test values: a = 2, b = 3, …; `-D` overrides; memory model `mem[x] = 1000 + x`. |
| `ir_div` | 391 | Division truncating toward zero; reports division by zero instead of crashing (used by `ir_eval` and both simulators). |
| `ir_eval` | 400 | Evaluates the tree directly. This is the reference value every generated program must match; its `ok` flag is cleared if the tree divides by zero. |

**Used by:** `main.c` (parse, print, evaluate), both simulators (variable values, memory).

## src/burs/target.h

82 lines · engine ↔ target interface **Utsav**

Describes what a target machine must provide. The engine only ever talks to targets through the `Target` struct defined here, which is what makes the system retargetable.

| Item | Line | What it is |
|---|---|---|
| `BURS_MAX_KIDS / _INSTR / _INSTR_LEN` | 18–20 | Limits: up to 512 instructions of 48 characters per program. |
| `PANIC` | 23 | Error hook required by the generated matcher. |
| `Operand` | 26 | What a reduced subtree turns into: `"r3"`, `"#4"`, `"[a]"` or top of stack. |
| `EmitCtx` | 31 | The output buffer: the instruction lines, the next free register, an overflow flag. |
| `RuleInfo` | 39 | One rule's printable pattern, cost and instruction template. |
| `struct Target` | 45 | Everything about one machine: name, start nonterminal, rule table, the four generated matcher functions, `reduce()` and `simulate()`. |
| `targetA`, `targetB` | 66–67 | The two targets, defined at the end of each `.gr` file. |
| helper prototypes | 70–80 | `burs_emit`, `burs_operand`, `burs_new_reg`, `burs_split` (implemented in `engine.c`). |

**Used by:** `engine.c`, `main.c`, both `.gr` files.

## src/burs/engine.h

30 lines · BURS engine API **Utsav**

Declares the six engine functions `main.c` calls (lines 13–28). Each takes a `const Target *`, so the same code works for any machine.

## src/burs/engine.c

205 lines · BURS engine **Utsav** **helpers: Ahmad**

The machine-independent BURS driver. It never names an instruction: it labels the tree through the target's generated matcher, reads costs and the cover back, and drives code emission.

| Function | Line | What it does |
|---|---|---|
| `burs_label` | 15 | Bottom-up labeling: calls the generated `label()`, which walks the tree in postorder and stores a state in each node (one table lookup per node). Returns whether the root can be covered. |
| `burs_cost` | 21 | Cost of the best way to compute nonterminal *nt* at a node: the rule's cost plus its children's costs. |
| `print_cover` / `burs_print_cover` | 38, 64 | Backtracks from (root, start nonterminal) and prints the selected rules top-down ("Selected cover"). |
| `cost_table` / `burs_print_cost_table` | 69, 96 | The −v table: for every node and nonterminal, the best rule and its cost. |
| `burs_reduce` | 103 | Code emission: for the selected rule, reduces the children first (left to right), then calls the target's `reduce()`. Output comes out in postorder. |
| `burs_print_rules` | 121 | Prints a target's rule table (`--rules`). |
| `burs_emit`, `burs_operand`, `burs_new_reg` | 138, 151, 162 | Helpers for emitters: append one instruction (stops at 512), build an operand string, allocate the next virtual register. |
| `burs_split` | 171 | Helper for simulators: splits `"ADDI r2,r1,#4"` into opcode and operands. |

**Calls:** the target's generated matcher and its `reduce()`. **Called by:** `main.c`.

## src/targets/targetA/targetA.gr

194 lines · Target A, register machine **rules: Mayank** **emitter + simulator: Ahmad**

Everything about Target A, a 3-address load/store machine (LOAD, LOADI, ADD, ADDI, SUB, SUBI, MUL, DIV, DIVI; unlimited virtual registers). The rules section is read by burg; the C after the second %% is copied into the generated file.

| Part | Line | What it does |
|---|---|---|
| preamble %{ … %} | 1–22 | Comment describing the ISA; includes target.h. |
| %start reg, %term … | 24–25 | Start nonterminal and the seven operators (same numbers as ir.h). |
| 11 rules | 28–38 | nonterminal: PATTERN = rule-number (cost); e.g. reg: ADD(reg,imm) = 6 (1); is ADDI; rules 10–11 are DIV and DIVI (cost 3). |
| ntA[], rulesA[] | 41, 47 | Nonterminal names; printable pattern, cost and template for each rule. |
| reduceA | 63 | The emitter: given one selected rule and its children's operands, emits that instruction into a new register and returns it. imm: CONST emits nothing and returns #k. |
| simfail, regnum | 101, 107 | Simulator helpers: format an error; parse rN. |
| simulateA | 120 | Runs the generated text on a register file, rejects undefined registers and division by zero, returns the result register's value. |
| const Target targetA | 186 | Packs all of the above (plus the generated burmA_* functions) into one object. |

## src/targets/targetB/targetB.gr

208 lines · Target B, stack machine **rules: Mayank** **emitter + simulator: Ahmad**

Everything about Target B, a 0-address stack machine with memory (ADDM [x]) and immediate (ADDI #k) operand forms. Same structure as Target A.

| Part | Line | What it does |
|---|---|---|
| %start stk, %term … | 24–25 | Start nonterminal and the same seven operators. |
| 17 rules | 28–44 | Nonterminals stk (on the stack), mem (a variable operand), imm (a constant). Chain rules stk: mem / stk: imm are PUSH / PUSHI; rules 15–17 are DIV, DIVM, DIVI. |
| ntB[], rulesB[] | 47, 54 | Nonterminal names and the printable rule table. |
| reduceB | 76 | The emitter: mem and imm return operands; PUSH rules push them; ADD/SUB/MUL/DIV work on the stack; …M and …I forms take the right operand directly. |
| operand_value, arith | 120, 138 | Simulator helpers: decode [x] / #k; do +, −, ×, ÷ (reporting division by zero). |
| simulateB | 148 | Runs the code on a value stack; checks underflow, operand kinds, division by zero, and that exactly one value is left. |
| const Target targetB | 200 | Packs Target B into one object. |

### `src/main/main.c`

575 lines · driver and test modes **Ahmad**

The `burs` program itself. It reads the command line, runs the expression through every selected target, prints the report, and provides the suite, exhaustive, benchmark and interactive modes.

| Function / item | Line | What it does |
|---|---|---|
| `EXIT_USAGE / _PARSE / _CHECK` | 15–17 | Exit codes 1, 2, 3 (0 = success). |
| `all_targets[]`, `suite[]` | 19, 23 | The targets to run (add a new target here) and the 11 review expressions. |
| `usage` | 53 | The –h help text. |
| `collect_vars`, `print_vars` | 73, 90 | The "Test values" line. |
| `run_target` | 115 | The pipeline for one target: label → cost table (–v) → cover → emit → print code → simulate → compare (skipped when the value is undefined). |
| `process` | 191 | One expression: parse, print the IR, compute the reference (or "undefined" on division by zero), run every target, print the retargeting summary. |
| `run_suite` | 243 | `--suite`: the 11 expressions as a PASS/FAIL table. |
| `naive_cost`, `run_exhaustive` | 292, 310 | `--exhaustive`: all 7,140 trees of depth ≤ 2 including DIV (294 divide by zero and are skipped) on both targets, plus the naive-selector comparison. |
| `random_tree`, `count_nodes`, `run_bench` | 390, 409, 414 | `--bench`: times labeling on random trees from 405 to 239,710 nodes. |
| `interactive` | 455 | `-i`: one expression per line until `quit`. |
| `set_var` | 480 | Handles `-D name=value`. |
| `main` | 495 | Parses options (`-t`, `-v`, `-D`, `-i`, `--rules`, `--suite`, `--exhaustive`, `--bench`) and picks the mode. |

## 3. Build, tests and results

| File | Owner | What it does |
|---|---|---|
| `Makefile` | Vineet | Targets: `make` (3-stage build into `build/`), `make test`, `make demo`, `make results`, `make clean`. |
| `tests/cases.txt` | Ahmad | 31 regression cases, one per line: `id \| options \| expression`. Grouped as positive (`pos_`), boundary (`bnd_`), rule choice (`rule_`), negative (`neg_`, exit 2) and failure (`fail_`, divide by zero, exit 3). |
| `tests/expected/<id>.out` | Ahmad | The exact output each case must produce, including `exit=N` on the last line. 31 files. |
| `tests/run_tests.sh` | Ahmad | Runs every case and compares its output byte for byte with the expected file, then runs `--suite` and `--exhaustive`. Prints `33 passed, 0 failed`. `--update` rewrites the expected files after a deliberate change. |
| `tests/make_results.sh` | Ahmad | Regenerates everything in `results/`. |
| `results/rule_tables.txt` | generated | Output of `--rules`: both rule tables. |
| `results/sample_runs.txt` | generated | Full –v output for the six review expressions and `(a + b) / c`. |
| `results/suite_summary.txt` | generated | Output of `--suite` (11/11 PASS). |
| `results/exhaustive_check.txt` | generated | Output of `--exhaustive` (13,692 runs, 0 failures; naive 8.49, A 8.05, B 6.72). |

| | | |
|---|---|---|
| `results/benchmark.txt` | generated | Output of `--bench` (time per node vs tree size). |
| `results/test_report.txt` | generated | Output of `run_tests.sh`: 33 PASS lines. |
| `README.md` | Ahmad | Project overview, commands, demo walkthrough, rule tables, results. |
| `.gitignore` | – | Keeps `build/` out of the repository. |

## 4. Documents (`docs/`)

| File | What it is |
|---|---|
| `Review1_Presentation.pptx`, `Review1_Document.docx` | Review 1: problem, design, plan. |
| `Review2_Document.pdf` | Review 2: implementation progress (Target A). |
| `Review3_Guidelines.pdf` | The course's Review 3 guidelines. |
| `Review3_Document.docx` / `.pdf` | Review 3 final report. |
| `Review3_Presentation.pptx` / `.pdf` | Review 3 slides with speaker notes. |
| `Review3_Presenter_Notes_Ahmad_Azim.pdf` | Ahmad's presenter notes and viva preparation. |
| `Code_Guide.pdf` | This document. |

## 5. Third-party: `third_party/burg/`

BURG is the code-generator generator by Fraser, Hanson and Proebsting, in Chen Zhexuan's modernised version. It is **not team code**: it is included unmodified so `make` can build it, and it runs only at build time to turn each `.gr` file into a C matcher. Its licence is in `COPYRIGHT` and `LICENSE.TXT`.

**Reads the grammar**

`main.c` (command line: `-p`, `-o`), `lex.c` (tokenizer), `gram.y` → `gram.tab.c/.h` (grammar parser generated by Bison), `fe.c`, `fe.h` (declarations, rules, patterns).

**Computes the BURS states**

`burs.c`, `item.c`, `closure.c` (chain rules), `delta.c` (cost normalisation), `map.c` (state map), `queue.c` (work list), `trim.c`, `table.c` (transition tables).

**Writes the C matcher**

`be.c` (writes `label`, `rule`, `kids`, `nts` and the tables), `plank.c` (compresses the tables).

**Utilities and other files**

`b.h`, `list.c`, `string.c`, `symtab.c`, `rule.c`, `pattern.c`, `operator.c`, `nonterminal.c`, `zalloc.c`; `sample.gr`, `doc.tex`, READMEs, `CMakeLists.txt` (unused by our Makefile).

## 6. "Where do I look if…?"

| You want to… | Look at |
|---|---|
| change an instruction's cost | the rule line in `targetA.gr` (28–38) or `targetB.gr` (28–44) *and* the matching `rulesA[]` / `rulesB[]` entry |
| change how an instruction is printed | `reduceA` (targetA.gr:63) or `reduceB` (targetB.gr:76) |
| change an error message for bad input | the parser in `ir.c` (213–340) |
| change test variable values or the memory model | `ir_var_value` / `ir_mem_read` in `ir.c` (370, 385) |
| change what the report prints | `run_target` / `process` in `main.c` (115, 191); cover and cost table in `engine.c` |

| | |
|---|---|
| add a command-line option | `main` in `main.c` (495) and `usage` (53) |
| add a test case | a line in `tests/cases.txt`, then `sh tests/run_tests.sh --update` and review the new `.out` file |
| add a new IR operator | `ir.h` enum + `ir.c` (parser, printer, `ir_eval`) + the `%term` line, rules, `reduce` and `simulate` cases in both grammars. Exactly how DIV was added; see commit "Add division (DIV)". |
| add a third target machine | new `src/targets/targetC/targetC.gr` (same `%term`; rules, `reduce`, `simulate`, `const Target targetC`), a `burg -p burmC` rule in the `Makefile`, `extern` in `target.h`, and `&targetC` in `all_targets[]` (main.c:19) |

## 7. Function calls for `./build/burs "a + b * c"`

```
main()                                      main.c:495   options → one expression
└─ process()                                main.c:191
   ├─ ir_parse()                             ir.c:310     text → tree, nodes numbered n1..n5
   ├─ printtree(), ir_draw(), print_vars()   ir.c / main.c
   ├─ ir_eval()                              ir.c:400     reference value = 14
   └─ for each target in all_targets[]:
      └─ run_target()                        main.c:115
         ├─ burs_label()                     engine.c:15  → burmA_label() / burmB_label() (generated)
         ├─ burs_cost()                      engine.c:21  total cover cost
         ├─ burs_print_cover()               engine.c:64  "Selected cover"
         ├─ burs_reduce()                    engine.c:103 children first, then…
         │  └─ reduceA() / reduceB()         targetX.gr   …emit one instruction per rule
         └─ simulateA() / simulateB()        targetX.gr   run the code; must equal 14
   └─ retargeting summary                    main.c       A: 5 instr, cost 6 · B: 4 instr, cost 5
```